API REST · NODE.JS + POSTGRESQL

Balsas Experience API

> Revisão da entrega: API publicada e verificada em 17/10/2026. Faltam o nome da equipe, matrículas e confirmação dos relatos individuais.

1. Identificação

Campo	Informação
Instituição	UNIBALSAS - Centro Universitário de Balsas
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Programação para Web
Professor	Marcos Deivd de Spusa Ramos (grafia informada)
Projeto	Balsas Experience API
Data do documento	17/10/2026

2. A equipe

Nome da equipe: **a informar**. Todos os integrantes precisam constar abaixo.

Contribuições sugeridas para revisão dos integrantes; ainda não confirmadas.

Nome completo	Matrícula	Contribuição proposta
Carinne Borges da Silva	A informar	Organização da documentação e descrição dos endpoints.
Júlio César Lima dos Reis	A informar	Desenvolvimento das rotas e validação dos dados recebidos.
João Pedro Strasser Santos	A informar	Modelagem do banco e integração com Prisma.
Rogério Mota de Melo	A informar	Configuração do ambiente, testes e publicação no Render.

3. Descrição do projeto

A Balsas Experience API disponibiliza um catálogo de atrações turísticas de Balsas-MA. Permite consultar atrações, filtrar por categoria, tag e texto, acessar detalhes e registrar avaliações de visitantes. Também oferece operações para criar, atualizar e remover atrações.

Serve como fonte de dados para sites e aplicativos voltados a moradores e turistas. O projeto reúne informações dispersas em uma estrutura consultável e persistente: categorias organizam o catálogo, tags permitem cruzar interesses e avaliações registram a experiência dos visitantes.

4. Tecnologias utilizadas

Versões de referência do ambiente local e dependências. A versão do banco publicada não foi consultada.

Tecnologia	Versão verificada	Finalidade
Node.js	24.13.1	Ambiente de execução JavaScript
Express	5.2.1	Rotas e respostas HTTP
Prisma CLI / Client	7.10.0	Modelagem, migrações e consultas
PostgreSQL	18.6	Banco de dados relacional
@prisma/adapter-pg	7.10.0	Adaptador Prisma para PostgreSQL
pg	8.23.1	Driver PostgreSQL
dotenv	18.0.5	Carregamento das variáveis de ambiente
npm	11.17.0	Dependências e scripts

5. Modelo de dados

PK = chave primária; FK = chave estrangeira. IDs inteiros são gerados automaticamente; campos sem indicação de opcional são obrigatórios.

Tabela	Campos e restrições
Categoria	id: Int, PK; nome: String, único
Tag	id: Int, PK; nome: String, único
Atracao	id: Int, PK; nome: String; descricao: String; cidade: String (padrão Balsas); estado: String (padrão MA); distanciaCentroKm: Int opcional; criadaEm: DateTime (agora); categoriaId: Int, FK → Categoria.id
Avaliacao	id: Int, PK; visitante: String; nota: Int; comentario: String opcional; criadaEm: DateTime (agora); atracaoId: Int, FK → Atracao.id
_AtracaoToTag	A: Int, FK → Atracao.id; B: Int, FK → Tag.id. PK composta (A, B). Tabela de associação gerenciada pelo Prisma.

Relações: Categoria 1:N Atracao; Atracao 1:N Avaliacao; Atracao N:N Tag. Excluir uma atração remove suas avaliações e associações com tags em cascata; categorias e tags são preservadas. A API valida notas inteiras entre 1 e 5.

Por que separar tags? Uma tabela própria permite reutilizar a mesma tag em várias atrações, evitar duplicação de nomes e aplicar filtros por relacionamento.

6. Endpoints da API

Método	Caminho	O que faz	Status
GET	/	Identificação do serviço	200
GET	/health	Estado do processo HTTP	200
GET	/atracoes	Listagem por nome; categoria e tags incluídas	200
GET	/atracoes?categoria=Lazer &tag=lazer&q=rio	Filtros combinados, sem diferenciar maiúsculas	200
GET	/atracoes/:id	Detalhes, avaliações, média e total	200 / 404
GET	/categorias	Categorias e contagem de atrações	200
POST	/atracoes	Criar atração	201 / 400 / 409
PUT	/atracoes/:id	Substituir dados e tags	200 / 400 / 404 / 409
DELETE	/atracoes/:id	Excluir atração e avaliações	204 / 404
POST	/atracoes/:id/avaliacoes	Criar avaliação de visitante	201 / 400 / 404

Falhas inesperadas podem retornar 500. /health não verifica o banco. POST e PUT exigem nome, descricao e categoria. Cidade, estado, distância e tags são opcionais. PUT substitui tags e restaura os padrões dos campos opcionais omitidos.

6. Endpoints da API · exemplo completo

Requisição e resposta reais do cliente HTTP Node.js/fetch, capturadas em ambiente local em 15/10/2026. O registro temporário foi removido após a coleta.

Requisição

```
POST /atracoes HTTP/1.1
Host: 127.0.0.1:3110
Content-Type: application/json

{
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "categoria": "Histórico e Urbano",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "tags": []
}
```

Resposta

```
HTTP/1.1 201 Created
Location: /atracoes/11
Content-Type: application/json; charset=utf-8

{
  "id": 11,
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "criadaEm": "2026-10-15T16:25:50.047Z",
  "categoriaId": 1,
  "categoria": {
    "id": 1,
    "nome": "Histórico e Urbano"
  },
  "tags": []
}
```

Busca inexistente: `GET /atracoes/2147483647` retorna 404 com `{"erro":"Atração não encontrada"}`. Remoção bem-sucedida retorna 204 sem corpo.

7. Como executar o projeto

Pré-requisitos: Node.js 24 LTS, npm, Git e acesso a um PostgreSQL local ou hospedado. Execute os comandos na raiz do repositório.

1. Clonar e instalar

```
git clone https://github.com/ark-pkr/balsas-experience-api.git
cd balsas-experience-api
npm ci
```

O repositório público está indicado na seção 9.

2. Configurar o ambiente

Copie .env.example para .env e substitua os exemplos pelas credenciais do seu banco. Não publique o .env.

```
DATABASE_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-POOLER/neondb?sslmode=require"
DIRECT_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-DIRETO/neondb?
sslmode=require&connect_timeout=30"
PORT=3000
```

A aplicação usa DATABASE_URL. O Prisma CLI usa DIRECT_URL ou, se ausente, DATABASE_URL. As senhas acima são fictícias.

3. Gerar cliente, migrar e popular

```
npx prisma generate --config prisma7.config.ts
npx prisma migrate deploy --config prisma7.config.ts
npm run seed
```

Atenção ao seed: apaga os dados existentes antes de inserir oito atrações e três avaliações. Use apenas em banco novo ou descartável; pule essa etapa se o banco já estiver populado.

4. Iniciar e verificar

```
npm start
# Em desenvolvimento, alternativamente: npm run dev
# Em outro terminal:
curl -i http://localhost:3000/health
curl -i http://localhost:3000/atracoes
npm test
```

No PowerShell, use curl.exe. Os testes usam a porta 3108 e criam/removem uma atração temporária. Os 15 cenários passaram na verificação local.

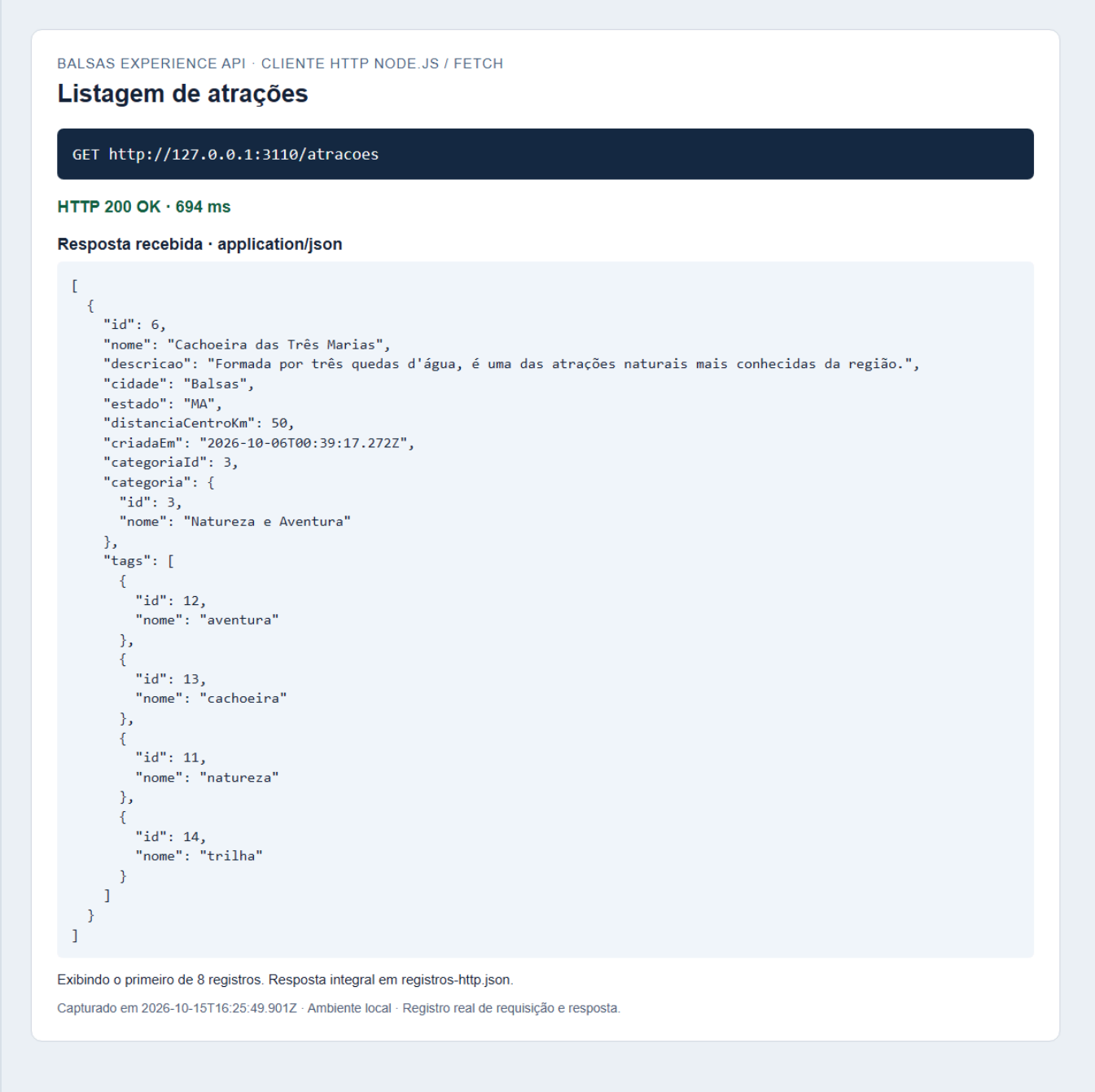
Publicação preparada

O arquivo render.yaml configura um Web Service no Render com Node.js 24.13.1. Build: `npm ci --include=dev && npm run build`; início: `npm start`. Cadastrar DATABASE_URL e DIRECT_URL no painel. O build aplica migrações e não executa seed. A hospedagem fornece PORT.

8. Evidências de funcionamento

Listagem de atrações · HTTP 200

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



BALSAS EXPERIENCE API · CLIENTE HTTP NODE.JS / FETCH

Listagem de atrações

```
GET http://127.0.0.1:3110/atracoes
```

HTTP 200 OK · 694 ms

Resposta recebida · application/json


```
[
  {
    "id": 6,
    "nome": "Cachoeira das Três Marias",
    "descricao": "Formada por três quedas d'água, é uma das atrações naturais mais conhecidas da região.",
    "cidade": "Balsas",
    "estado": "MA",
    "distanciaCentroKm": 50,
    "criadaEm": "2026-10-06T00:39:17.272Z",
    "categoriaId": 3,
    "categoria": {
      "id": 3,
      "nome": "Natureza e Aventura"
    },
    "tags": [
      {
        "id": 12,
        "nome": "aventura"
      },
      {
        "id": 13,
        "nome": "cachoeira"
      },
      {
        "id": 11,
        "nome": "natureza"
      },
      {
        "id": 14,
        "nome": "trilha"
      }
    ]
  }
]
```


Exibindo o primeiro de 8 registros. Resposta integral em registros-http.json.

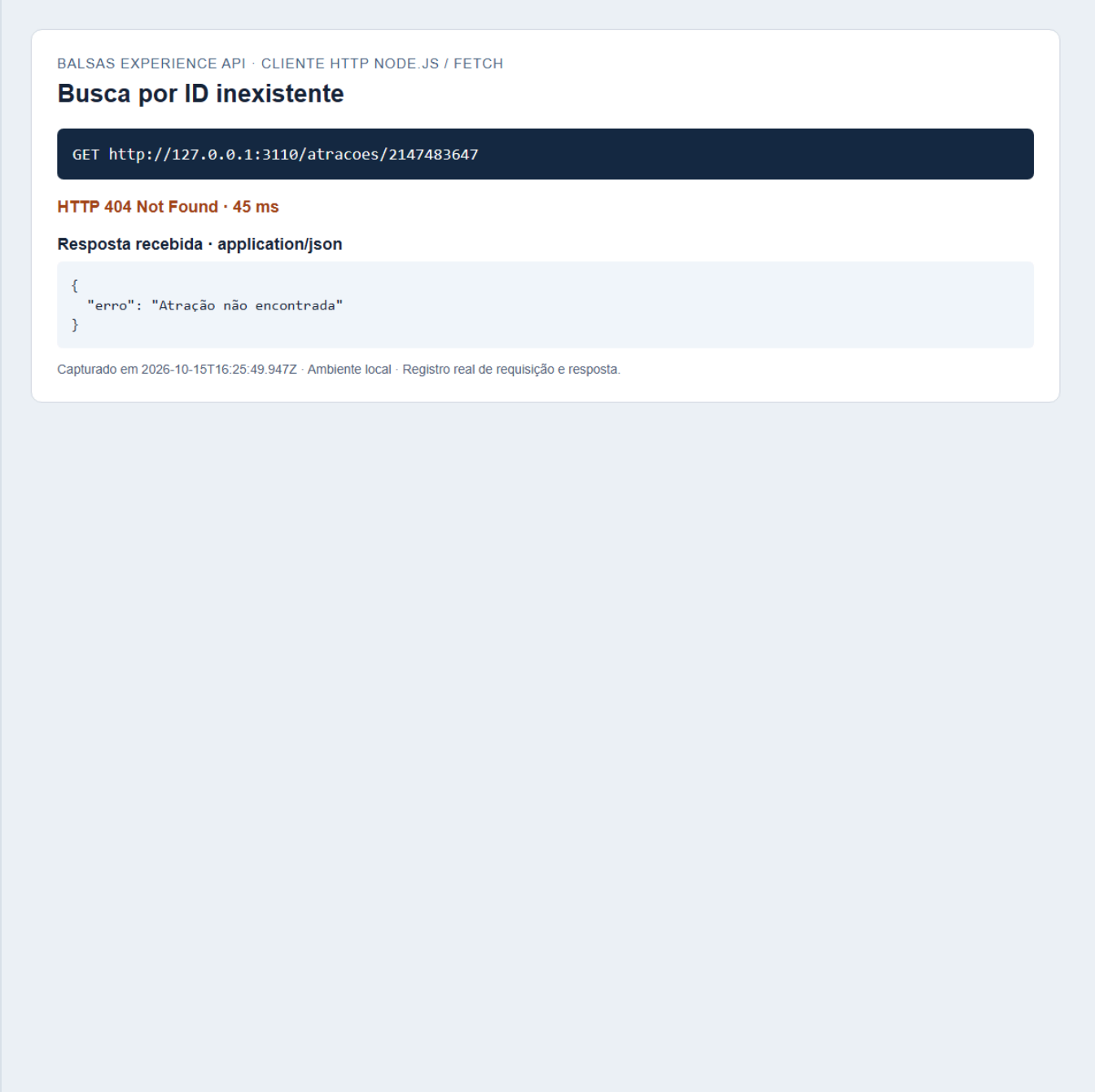
Capturado em 2026-10-15T16:25:49.901Z · Ambiente local · Registro real de requisição e resposta.

GET http://127.0.0.1:3110/atracoes · Status recebido: 200. A captura mostra o primeiro item da listagem; o JSON integral foi preservado.

8. Evidências de funcionamento · continuação

Busca por ID inexistente · HTTP 404

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.

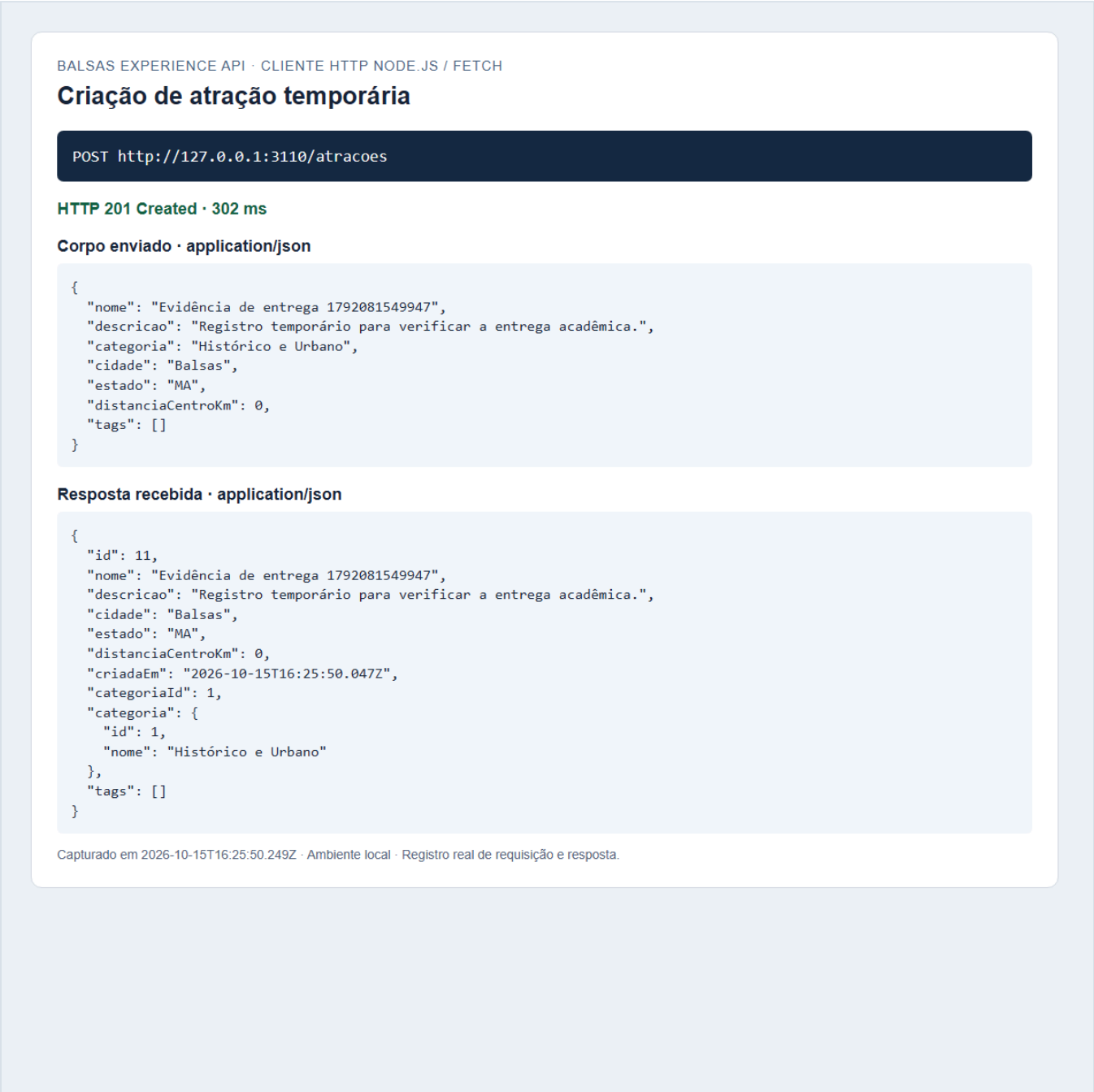


GET http://127.0.0.1:3110/atracoes/2147483647 · Status recebido: 404.

8. Evidências de funcionamento · continuação

Criação de atração temporária · HTTP 201

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



BALSAS EXPERIENCE API · CLIENTE HTTP NODE.JS / FETCH

Criação de atração temporária

POST http://127.0.0.1:3110/atracoes

HTTP 201 Created · 302 ms

Corpo enviado · application/json

```
{
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "categoria": "Histórico e Urbano",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "tags": []
}
```

Resposta recebida · application/json

```
{
  "id": 11,
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "criadaEm": "2026-10-15T16:25:50.047Z",
  "categoriaId": 1,
  "categoria": {
    "id": 1,
    "nome": "Histórico e Urbano"
  },
  "tags": []
}
```

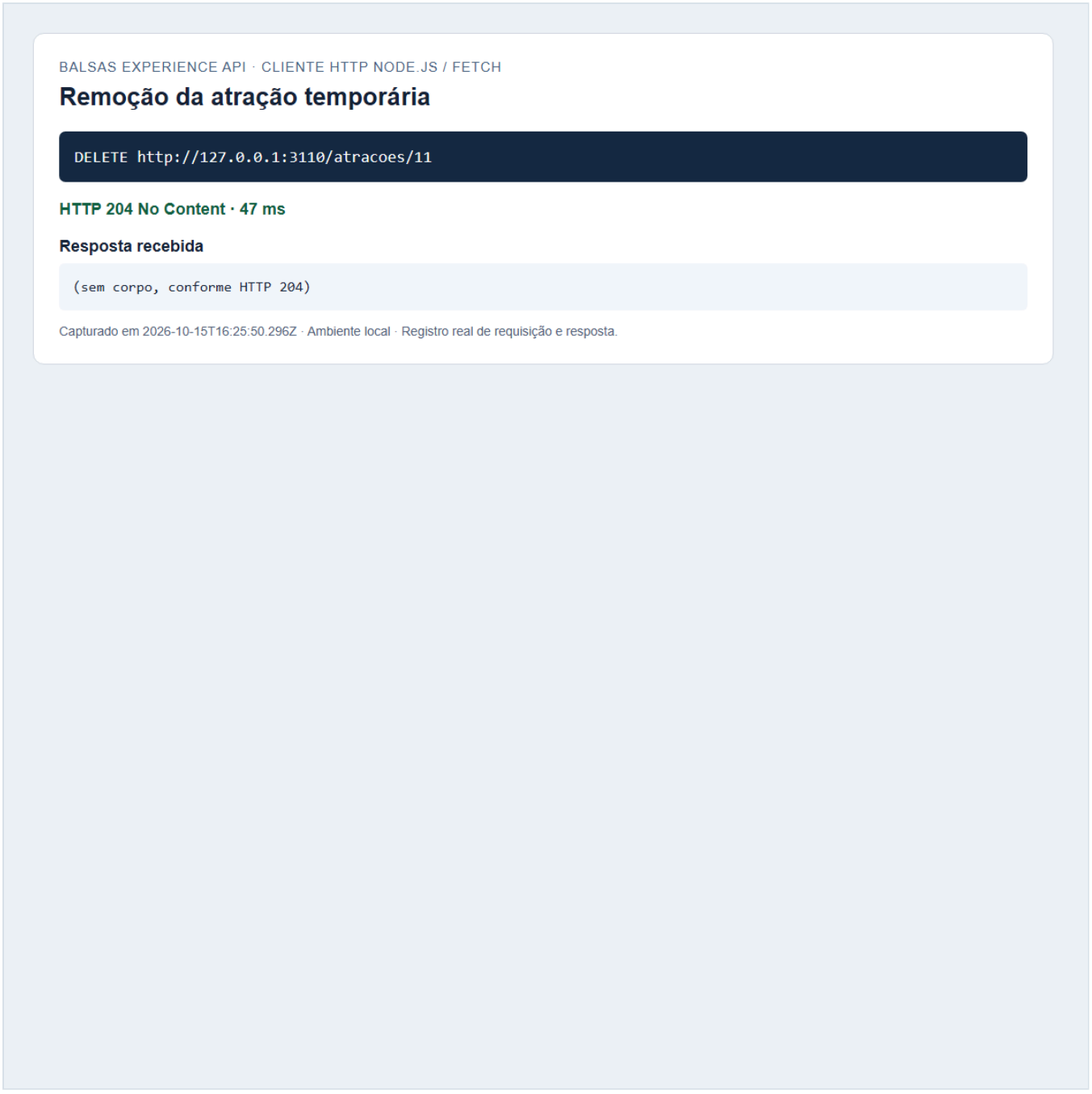
Capturado em 2026-10-15T16:25:50.249Z · Ambiente local · Registro real de requisição e resposta.

POST http://127.0.0.1:3110/atracoes · Status recebido: 201.

8. Evidências de funcionamento · continuação

Remoção da atração temporária · HTTP 204

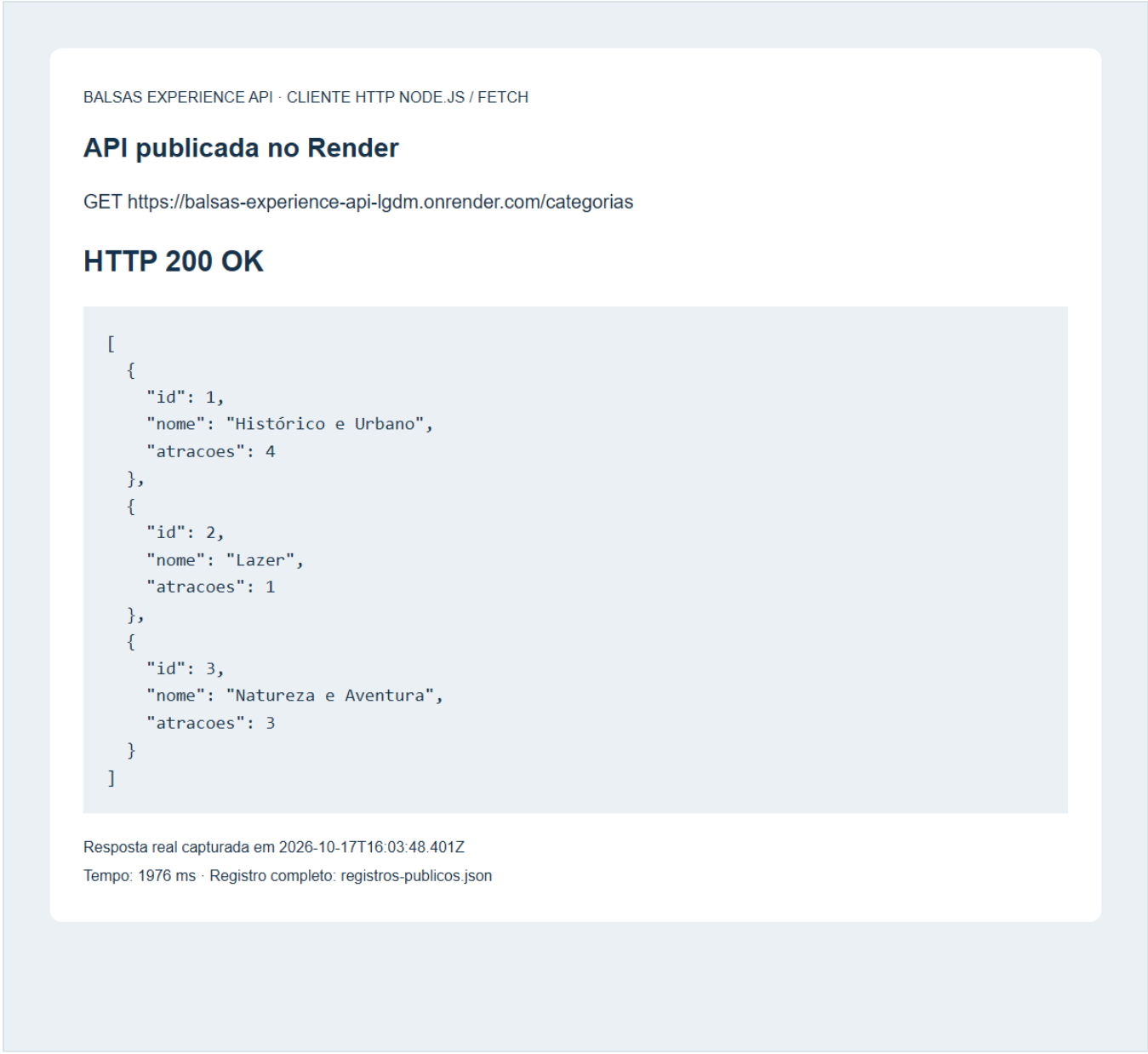
Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



DELETE http://127.0.0.1:3110/atracoes/11 · Status recebido: 204.

8. Evidências de funcionamento · API publicada

Verificação real em 17/10/2026: GET /health, /categorias e /atracoes retornaram HTTP 200; GET /atracoes/2147483647 retornou 404. As consultas de categorias e atrações confirmam o acesso ao banco publicado.



Captura da visualização da resposta real do cliente HTTP Node.js/fetch. Registros integrais: docs/evidencias/registros-publicos.json.

9. Links

Destino	Endereço
Repositório público no GitHub	https://github.com/ark-pkr/balsas-experience-api
API publicada	https://balsas-experience-api-lgdm.onrender.com/categorias

10. Dificuldades e aprendizados

Sugestões para revisão da equipe. Cada integrante deve ajustar e confirmar seu relato conforme sua experiência real.

Integrante	Dificuldade sugerida	Solução sugerida	Aprendizado sugerido
Carinne Borges da Silva	Entender os códigos de status HTTP.	Revisou as respostas durante os testes para documentá-las.	Documentar requisições, respostas e comportamentos da API.
Júlio César Lima dos Reis	Validar os campos enviados em JSON.	Testou entradas válidas e inválidas e ajustou as verificações.	Criar rotas no Express e retornar mensagens de erro adequadas.
João Pedro Strasser Santos	Compreender as relações entre atrações, categorias, tags e avaliações.	Revisou o schema e conferiu os vínculos no banco.	Modelar relações e utilizar migrações e consultas com Prisma.
Rogério Mota de Melo	Configurar a conexão com PostgreSQL e as variáveis de ambiente na hospedagem.	Conferiu as configurações e testou os endpoints publicados.	Publicar uma API e verificar a comunicação entre servidor e banco.

Para cada integrante: descreva um problema concreto, a tentativa que resolveu o problema e o conhecimento adquirido. Exemplos de assuntos que podem ser abordados, se realmente ocorreram: conexão com PostgreSQL, variáveis de ambiente, adaptação ao Prisma 7, relações entre tabelas, migrações, validação de JSON e publicação.

Conceitos técnicos demonstrados pelo projeto

Prisma 7: o cliente usa o adaptador PrismaPg para acessar PostgreSQL. **Modelagem:** tags reutilizáveis são associadas às atrações em uma relação N:N. **Integridade:** a exclusão em cascata evita avaliações sem atração. **HTTP:** criação retorna 201 e Location; remoção retorna 204 sem corpo; registro ausente retorna 404.

Esses conceitos são verificáveis no código; a experiência pessoal de cada integrante será inserida após a confirmação em docs/RELATOS-EQUIPE.md.

Verificação técnica realizada nesta revisão

A revisão local anterior registrou 15 cenários automatizados aprovados. Foram coletados registros reais de listagem 200, busca 404, criação 201 e remoção 204, usando o banco PostgreSQL 18.6 configurado. O seed não foi executado nesta revisão.

Antes de enviar pelo portal

1. Confirmar a identificação acadêmica, a grafia do professor e as contribuições individuais.
2. Conferir os links públicos da API e do repositório.
3. Confirmar os aprendizados reais de cada integrante e as matrículas.
4. Gerar novamente o PDF e nomeá-lo balsas-experience-api-equipe-<nome-da-equipe>.pdf.
5. Combinar quem será o único integrante responsável pela submissão e enviar somente o PDF.